

Relatorio

June 17, 2026

Contents

1 Sobre a organização dos arquivos:	1
2 Funcionamento	2
2.1 Estruturas	2
2.2 Game-loop	3
2.2.1 São 5 casos:	3
2.3 Um pouco mais de perto (cases)	4
2.3.1 Renderização	4
2.3.2 O principal (case 1)	4

1 Sobre a organização dos arquivos:

Os principais arquivos são esses: A estratégia foi agrupar atividades semelhantes em cada arquivo. Cada header file (.h) inclui o protótipo das funções declaradas no respectivo source file (.c)

```
tree | grep -e '.c' -e '.h'
```

└─ game.c	Definição das estruturas e funções que representam inimigos, jogador, sua movimentação, seus status
└─ game.h	
└─ main.c	Arquivo principal, onde se definem as variáveis e estruturas além do game-loop
└─ mapa.c	Trata do vetor de char que representa a posição inicial de todos elementos do mapa, a partir dos arquivos mapaX.txt
└─ mapa.h	
└─ menu.c	Trata dos diferentes menus e suas funcionalidades, incluindo suas opções e aparência.
└─ menu.h	
└─ placar.bin	Guarda os 10 melhores scores
└─ render.c	Lida com a renderização do mapa e todas entidades.
└─ render.h	

2 Funcionamento

2.1 Estruturas

De um modo geral, foram definidas estruturas que representam inimigos (flames), o jogador (player) além de um tipo base (base); Essas estruturas contém atributos básicos como posição, velocidade além de flags como “está vivo?”, “está pindurado?”; Essas estruturas são utilizadas majoritariamente na implementação da jogabilidade do game-loop, isto é, são utilizadas na renderização das entidades (inimigos e jogador) no mapa durante o jogo, bem como sua movimentação, interações entre si (colisão entre entidades, por exemplo, matar o player caso toque em um inimigo) e entre o mapa (Logica que previne entidades de penetrar as plataformas). Além dessas (que são as principais), foram utilizadas estruturas auxiliares: “entities” que funciona como vetor contendo todas entidades ativas, e a estrutura “base”; Essa ultima contém todos os campos que são comuns a jogadores e inimigos (principalmente referentes a posição e locomoção): Todos os campos comuns a essas tres (inimigos, jogador e base) são declarados na mesma ordem e no inicio de cada tipo; Isso permite que criemos funções mais genericas, que recebam de parametro tanto inimigos ou jogadores, utilizando um cast para esse tipo

base.

- Por exemplo, temos a função void `calculaCantosInt2(base* entity);`

podemos chama-la para atualizar os cantos (que são campos dos jogadores e inimigos) de entidades:

```
calculaCantosInt2((base*)jogador);  
ou  
calculaCantosInt2((base*)inimigo);
```

2.2 Game-loop

Dentro de `main.c`, o game-loop se divide em algumas subpartes, cada uma sendo um 'case' em um switch case principal:

2.2.1 São 5 casos:

1. 'case 0' define a representação e funcionalidade do menu inicial;
 2. 'case 1' define a principal parte do loop, que trata do 'jogo rodando'; É aqui que o jogador pode se locomover, saltar, ser morto pelos inimigos...
 3. 'case 2' faz funcionar a tela "você morreu", quando o jogador, more...
 4. 'case 3' a tela de transição quando o jogador passa de fase
 5. 'case 4' a tela de Pause;
- Obs: os 'case's 0 e 4 apresentam menus interativos, enquanto 2 e 3 apenas mostram mensagens;
 - A cada game-loop, uma flag chamada 'gameMode' decide qual dos switch cases será executado, sendo ela alterada apropriadamente em cada situação.

2.3 Um pouco mais de perto (cases)

2.3.1 Renderização

De uma forma bem geral, em cada um dos casos, a ultima coisa a ser feita é a renderização, que ocorre no escopo de "Drawing()"; Invariavelmente, isso segue a lógica de primeiramente, tornar tudo preto (ClearBackground(BLACK);) e depois se desenha o que deve ser visto; Isso se repete todo frame (60 vezes por segundo) Ex:

```
BeginDrawing();  
    ClearBackground(BLACK);  
    menuDraw(1,menu,ptrStatus);  
EndDrawing();
```

Isso é verdade em todos os casos; Uma função decide o que será desenhado, (no caso acima "menuDraw(1,menu,ptrStatus);")

- **Tangente, render.c**

Apesar do esforço para separar o código em arquivos de semelhantes funcionalidades, render.c acaba lidando apenas com uma função (e bem simples), responsável pela renderização no caso 'case 1' de jogo; No entanto, o restante das funções de renderização ficam localizadas no menu.c.

2.3.2 O principal (case 1)

Esse é de longe o caso mais complexo. Ele pode ser subdividido em trechos:

1. 'ouvir' por inputs / mover entidades De um modo geral, nessa parte inicial, são registrados inputs via teclado, tanto sobre movimentação quanto para pausar o jogo. No primeiro caso, geralmente, atualiza-se o struct do jogador apropriadamente (ex. sua posição horizontal é acrescida caso o jogador aperte 'D'). Outras atualizações acometem todas entidades nesse estado, como a gravidade que altera os campos de velocidade vertical de todas entidades presentes, além da IA dos inimigos, que altera a posição dos inimigos... No segundo caso, caso o jogador aperte 'P', a flag gameMode é alterada, de modo que no frame seguinte o estado do jogo passa para o switch case apropriado.

2. calcular colisões e checar outros eventos Em geral, isso se dá checando se a entidade em questão “invadiu” os limites de um bloco sólido (as plataformas por exemplo), teleportando tal entidade de volta; Como a logica detecta essa invasão antes de renderizar (além do fato de serem 60 FPS), ocorre a ilusão de que o mapa é sólido; Outro caso semelhante é o das escadas, em que detecta-se a proximidade da entidade com o inicio ou fim das escadas, ou do jogador com o Fim da fase.
3. Renderizar